

Metodyka Scrum a Agile – różnice, przewodnik i wdrożenie w IT

07-01-2026

Piotr Kujawa

Agile czy Scrum – czy to jedno i to samo? Nie do końca, choć wiele firm używa tych nazw zamiennie albo je łączy, mówiąc na przykład o „metodyce Agile”. To błąd i warto o tym wiedzieć – zwłaszcza jeśli planujesz wdrożenie nowej struktury pracy w swoim zespole. W tym artykule wyjaśnimy dokładniej, czym się różni Agile od Scruma, pokażemy ich główne założenia i podpowiemy, kiedy stosować Scrum w projektach IT.

Spis treści

- [Co to jest Agile? Historia i Manifest](#)
- [Scrum a Agile – czy to to samo? Wyjaśniamy różnice](#)
- [Co to jest Scrum? Filary i teoria](#)
- [Zespół scrumowy – 3 kluczowe odpowiedzialności](#)
- [Cykl życia sprintu – jak wygląda proces krok po kroku?](#)
- [Artefakty w Scrumie – narzędzia kontroli](#)
- [Wdrożenie Scrum w projekcie IT – okiem praktyka](#)
- [Podsumowanie: czy Twoja firma jest gotowa na Agile?](#)

Co to jest Agile? Historia i Manifest

Agile to zbiór zasad „zwinnego zarządzania” projektami IT – choć da się je przenieść na inne branże – który stawia na pierwszym miejscu elastyczność na zmieniające się w trakcie projektu potrzeby klienta.

Aby lepiej zrozumieć, o co chodzi, warto cofnąć się do lutego 2001 roku. W kurorcie narciarskim Snowbird w Utah spotkało się 17 programistów, którzy, krótko mówiąc, mieli dość tradycyjnych, kaskadowych metod zarządzania projektami (dziś określa się je metodykami Waterfall). Wymagały miesięcy planowania w przód, utrzymywania bardzo szczegółowej dokumentacji i sztywnego trzymania się harmonogramu, ale, i to jest

najważniejsze, **nie pozostawiały miejsca na większe modyfikacje założeń projektu już w trakcie realizacji.**

Pewnego wieczoru opracowali więc [Manifest Agile](#) – spis fundamentalnych wartości, które stały się później punktem wyjścia dla nowoczesnych metod tworzenia oprogramowania. Brzmiały one tak:

1. **Ludzie i interakcje ponad procesy i narzędzia.**
2. **Działające oprogramowanie ponad obszerną dokumentację.**
3. **Współpraca z klientem ponad formalne ustalenia.**
4. **Reagowanie na zmiany ponad podążaniem za planem.**

Do tego dochodzi 12 bardziej szczegółowych zasad, które rozwijają te wartości. Mówią one m.in. o ciągłym dostarczaniu wartości klientowi, akceptowaniu zmieniających się wymagań (nawet na bardzo późnych etapach projektu) czy o ścisłej współpracy między deweloperami a „biznesową” częścią zespołu.

Scrum a Agile – czy to to samo? Wyjaśniamy różnice

Agile samo w sobie nie jest metodą projektową, tylko pewną filozofią, sposobem myślenia o tym, jak powinny wyglądać projekty. Na bazie tej idei powstały konkretne metody pracy, z własnymi zasadami oraz schematami projektowymi. Należą do nich m.in. Kanban, Crystal, Extreme Programming (XP)... i właśnie Scrum.

Ten ostatni jest dla wielu synonimem Agile i trudno się temu dziwić. W osiemnastej już edycji *State of Agile Report* z 2023 r. - badania publikowanego przez Digital.ai – można przeczytać, że **aż 63% zespołów IT, które opierają się na filozofii Agile, pracuje w Scrumie.** Ale mimo wszystko warto to rozróżnić.

	Agile	Scrum
Co to jest?	Filozofia zarządzania projektami	Praktyczna metoda pracy nad projektami, która wykorzystuje zasady Agile w biznesie
Z czego się składa?	4 wartości i 12 zasad z Manifestu Agile	Jasno zdefiniowane elementy, które tworzą ramy projektu: odpowiedzialności, wydarzenia i artefakty
Jak bardzo jest elastyczny?	Daje bardzo ogólne wytyczne, które można interpretować w różny sposób	Narzuca projektowi konkretną strukturę, której trzeba się trzymać. Można, oczywiście, zmienić np. długość sprintu, ale nie da się „zrobić Scruma” bez samych sprintów

Co to jest Scrum? Filary i teoria

Metodykę Scrum najprościej można opisać jako **framework dla projektów, który dzieli rozwój produktu na iteracje – sprinty**. Po każdym sprincie zespół projektowy powinien przedstawić albo działającą wersję produktu (np. aplikacji), albo przynajmniej jakąś konkretną funkcję.

Za twórców Scruma uważa się Kena Schwabera i Jeffa Sutherlanda – to właśnie oni są m.in. autorami [Scrum Guide](#), czyli oficjalnego przewodnika, w którym znajdziesz wszystkie zasady pracy w tej metodzie.

W podręczniku można więc przeczytać, że **Scrum jest frameworkiem empirycznym**. To znaczy, że decyzje w projekcie podejmuje się nie na podstawie planów czy założeń rozpisanych przed rozpoczęciem realizacji, tylko na bieżąco, na bazie tego, co faktycznie dzieje się z produktem. Na pierwszy rzut oka może to wyglądać jak przepis na kompletny chaos, ale tak nie jest – pod warunkiem że zespół trzyma się trzech filarów:

1. **Transparency** – każdy członek zespołu ma dostęp do wszystkich informacji o projekcie.
2. **Inspection** – zespół regularnie sprawdza postępy w realizacji celu, tak aby jak najszybciej wykryć problemy; służą temu wydarzenia oraz artefakty, o których więcej za moment.
3. **Adaptation** – jeśli zespół odkryje, że coś nie działa, że projekt idzie w innym kierunku, niż oczekuje klient, musi wprowadzić zmiany: albo w produkcie, albo w samym procesie jego realizacji.

Zespół scrumowy – 3 kluczowe odpowiedzialności

Zgodnie z przewodnikiem zespół nie powinien być większy niż 10 osób i każdy musi mieć przypisaną sobie „odpowiedzialność”. Wcześniej mówiono o rolach w Scrumie, ale w najnowszej wersji z 2020 r. to zmieniono, aby bardziej zaakcentować, że chodzi tu o konkretne zadania, za które dana osoba jest odpowiedzialna, a nie o tytuły czy hierarchię.

- **Product Owner** – określa cele i priorytety projektu oraz poszczególnych sprintów, zarządza backlogiem produktu, a do tego jest pośrednikiem między zespołem a klientem lub managementem firmy;
- **Scrum Master** – czuwa nad przestrzeganiem zasad Scrum; prowadzi spotkania, zajmuje się coachingiem zespołu przed rozpoczęciem projektu, dba też o to, aby zespół miał jak najlepsze warunki do pracy (np. jeśli deweloperzy zgłoszą, że wymagania od klienta są niejasne, to właśnie Scrum Master powinien je wyjaśnić);

- **Deweloperzy** – rozwijają produkt. I chodzi tu nie tylko o programistów, ale też o UX designerów, testerów czy devopsów. W modelowym Scrumie – to bardzo ważne – zespół deweloperski sam planuje codzienną pracę i dzieli zadania w ramach sprintu; liczy się to, czy na koniec dowiozą założony cel.

Cykl życia sprintu – jak wygląda proces krok po kroku?

Od początku przewija się tu pojęcie sprintu; tak nazywamy **jeden, zamknięty cykl pracy**. Sprints w Scrumie mogą trwać od 1 do 4 tygodni, w zależności od tego, jak złożony jest projekt, i mają swój stały rytm – w ramach każdego sprintu odbywa się kilka wydarzeń (eventów), które składają się na typową dla Scruma strukturę.

1. **Sprint Planning** – czyli planowanie sprintu. Cały zespół spotyka się na kilka godzin i ustala, jaki będzie cel na najbliższe tygodnie i które konkretnie zadania z backlogu da się w tym czasie zrealizować.
2. **Daily Scrum** – każdego dnia sprintu deweloperzy organizują maksymalnie kilkunastominutowe spotkanie, aby zsynchronizować ze sobą pracę. W najbardziej klasycznym podejściu wszyscy odpowiadają na trzy pytania:
 - 1) co zrobiłem wczoraj?
 - 2) co zrobię dzisiaj?
 - 3) jakie widzę przeszkody?,
 ale zespół ma dużą swobodę i może sam ustalić plan *Daily*.
3. **Sprint Review** – na koniec sprintu zespół przedstawia, co udało się osiągnąć w ostatnich tygodniach (konkretne funkcje, a nie prezentację w PowerPointcie!), a potem zbiera feedback od klienta i/lub managementu.
4. **Retrospective** – bardziej „wewnętrzne” podsumowanie sprintu, podczas którego omawia się, co poszło dobrze i co warto poprawić w kolejnych sprintach.

Artefakty w Scrumie – narzędzia kontroli

Aby usystematyzować pracę i ułatwić wszystkim członkom zespołu adaptację do warunków projektu, Scrum przewiduje również trzy *artefakty*. W skrócie, są to narzędzia, które mają obrazować, co jest do zrobienia, nad czym obecnie pracuje zespół i co zostało już wykonane.

- **Backlog produktu** to przejrzysta lista zadań koncentrujących się na wprowadzeniu, rozwijaniu i weryfikowaniu produktu. Zarządza nią Product Owner, ale wgląd do niej mają wszyscy członkowie zespołu;
- **backlog sprintu** to z kolei lista aktualnych zadań do wykonania w najbliższym sprincie, którą przygotowują deweloperzy;
- **product increment**, czyli „przyrost”; w *Scrum Guide* oznacza „sumę ukończonych elementów produktu podczas ostatniego sprintu i wszystkich

poprzednich”. A prościej można powiedzieć, że jest to... najnowsza działająca wersja produktu, którą da się zaprezentować na *Sprint Review*.

Co ważne, każdy artefakt powinien mieć swój konkretny punkt odniesienia.

- Dla backlogu produktu będzie to długoterminowy cel całego projektu (*Product Goal*), na przykład „Stworzyć platformę do zarządzania projektami dla małych zespołów, która pozwoli zastąpić arkusze w Excelu”;
- dla backlogu sprintu jest to po prostu cel danego sprintu (*Sprint Goal*);
- dla *incrementu*, tak zwane *Definition of Done*, czyli ustalone przez zespół kryteria techniczne, po których spełnieniu można stwierdzić, że dana iteracja produktu jest gotowa do działania.

Wdrożenie Scrum w projekcie IT – okiem praktyka

Z doświadczenia wiemy, że framework Scrum potrafi być efektywniejszy niż jakakolwiek inna metoda projektowa, przynajmniej w *developmentie*.

Ale nie zawsze! Po pierwsze, **nie wszystkie projekty w ogóle potrzebują takiego podejścia**. Jeśli produkt ma całkiem prosty backlog, który nie powinien się zmienić od startu do mety projektu, wtedy śmiało można poprowadzić projekt bardziej liniowo, tak jak w Waterfall.

A po drugie: **nie każdy zespół jest gotowy na „transformację Agile” i wdrożenie takiej filozofii**. W przytaczanym już wcześniej State of Agile Report z 2023 r. pojawił się ciekawy wniosek – główną barierą, która staje na drodze firmom chcącym przejść na zwinniejsze zarządzanie, jest „kultura organizacyjna sprzeczna z wartościami Agile”; tak przyznało 40% managerów. W wielu firmach pojawiają się problemy z:

- **brakiem zrozumienia, czemu służą scrumowe eventy** – na przykład *Daily Scrums* zamieniają się w długie, prawie godzinne sesje, podczas których każdy sucho raportuje managerowi, jakie taski wykonał dzień wcześniej (ma to nawet swoją nazwę, „*Zombie Scrum*”);
- **traktowaniem Scrum Mastera jak Project Managera albo... sekretarki zespołu** – zajmuje się wypełnianiem tabel w Jirze, *micromanagementem* i innymi zadaniami, które nie należą do obowiązków SMa;
- **brakiem decyzyjności po stronie Product Ownera** – co niemal zawsze kończy się kompletnym chaosem w projekcie.